Xebia — Principal Architect

L3 Techno-Managerial Round Interview Preparation

> **Round:** L3 — Techno-Managerial Final | **Interviewer:** Director / VP / Practice Head / Delivery Head **Format:** 60–90 min | Management + Technical Hybrid + Situational Judgement **Builds on:** L1 — 70 Q&As + L2 — Design + Consulting Deep-Dive **What L3 tests:** Can you lead, manage, AND stay technical? Can Xebia trust you with a $2M engagement?

What Makes L3 Different From L1 and L2

L1 (Technical)	L2 (Architecture + Consulting)	L3 (Techno-Managerial)
"How does Istio mTLS work?"	"Design a multi-region SaaS"	"Your 15-person team has 3 underperformers mid-engagement. What do you do?"
Right answers	Right thinking	Right decisions under pressure
Prove you know	Prove you can design	Prove you can **deliver + lead + grow revenue**
Individual contributor depth	Architecture vision	P&L awareness, team building, conflict resolution

L3 is where Xebia decides: "*Can we put this person in front of a Fortune 500 CTO, hand them a ₹15 Cr engagement, and sleep well at night?*"

Table of Contents

---

1. Engagement & Delivery Management

> These questions test whether you can **own the outcome** of a multi-million-dollar consulting engagement — not just the architecture diagram.

Q1: "You're the Principal Architect leading a 6-month cloud transformation engagement for a major Indian bank. Walk me through how you'd structure and run the engagement."

Answer:

> "I've run similar engagements, and I follow a structured playbook that balances delivery rigor with consulting agility:

Engagement Structure:

```
PHASE 1: MOBILIZATION (Week 1–2)
├── Governance Setup:
│   ├── RACI matrix: Who Decides, Who Executes, Who is Informed
│   ├── Steering committee: CTO/VP + Xebia Delivery Lead + Me (bi-weekly)
│   ├── Working committee: My team + client architects (weekly)
│   └── War room protocol: instant escalation for P1 issues
│
├── Team Onboarding:
│   ├── Client ecosystem: org chart, political landscape, decision-makers
│   ├── Technical landscape: existing infra, code repos, current pain points
│   ├── Compliance: NDA, security onboarding, access provisioning
│   └── Ways of working: tools (Jira/Confluence/Slack), ceremonies, code
standards
│
└── Delivery Plan:
    ├── Milestone-based roadmap (not sprint-based — steering committee
```

```
thinks in milestones)
    ├── 5 milestones mapped to SOW deliverables
    ├── Each milestone has: acceptance criteria, demo, sign-off
    └── Risk register: top 10 risks with mitigation owners

PHASE 2: DISCOVERY & FOUNDATION (Week 3–6)
├── Technical assessment + current state architecture
├── Target state architecture (validated with client architects)
├── PoC on ONE critical component (prove technology choices)
├── Platform foundation: IaC, CI/CD, observability baseline
└── Milestone 1 sign-off: "Foundation validated, ready for migration"

PHASE 3: BUILD & MIGRATE (Week 7–18)
├── 3 migration waves (sequenced by risk and value)
├── 2-week sprints with demos to working committee
├── Monthly demos to steering committee (CTO-level)
├── Architecture Decision Records (ADRs) for every major decision
├── Continuous integration with security scanning
└── Milestones 2–4 sign-offs at wave completions

PHASE 4: HARDEN & TRANSITION (Week 19–24)
├── Performance testing, security audit, DR testing
├── Production deployment with canary rollout
├── Knowledge transfer: 2-week handover to client's team
├── Documentation: architecture docs, runbooks, training materials
├── Milestone 5 sign-off: "Production live, team self-sufficient"
└── Transition plan: define next engagement scope (upsell)
```

What I track weekly:

Metric	Target	Why
Sprint velocity	Stable ± 15%	Predictability for stakeholders
Blocker age	< 48 hours	Unresolved blockers kill morale
Risk register changes	< 3 new risks/week	Situational awareness
Client satisfaction (pulse)	NPS > 50	Early warning if trust erodes
Team utilization	75–85%	Below = bench cost; above = burnout
Scope change requests	Tracked, not blocked	Manage, don't resist change

> **The difference between a good architect and a good engagement lead:** A good architect designs the right system. A good engagement lead ensures the right system gets delivered, accepted, and paid for."

Q2: "You're running a 15-person team across 2 time zones (India + US). The client says deliverables are slipping. How do you get back on track?"

Answer:

> "Slipping deliverables have a root cause — the first mistake is throwing more people at it without diagnosing why.

My diagnostic framework (FIRST 48 HOURS):

```
ROOT CAUSE ANALYSIS:
├── 1. SCOPE CREEP? (most common)
│   → Compare: what did the SOW say vs. what we're actually building?
│   → If scope grew 30%, timeline should have grown 30% too
│   → Action: Change request with revised timeline + cost. Present options,
not problems.
│
├── 2. TECHNICAL BLOCKERS?
│   → Are developers waiting for environments? Access? Client decisions?
│   → Review: average blocker resolution time (should be < 2 days)
│   → Action: Daily 15-min standup with blockers-only agenda. Escalate
unresolved in 24 hours.
│
├── 3. TEAM CAPACITY/SKILL MISMATCH?
│   → Is one team member doing 40% of the work? (bus factor = 1)
│   → Do we have senior developers doing junior-level tasks?
│   → Action: Re-balance work. Pair struggling members with strong ones. If
skill gap is structural, bring in specialist short-term.
│
├── 4. COMMUNICATION OVERHEAD? (Cross-timezone trap)
│   → Are India-US handoffs losing 24 hours per question?
│   → Action: 2-hour overlap window (9:30-11:30 PM IST / 12:00-2:00 PM EST)
│   │   Define this as sacred — NO meetings outside this for cross-timezone
work
│   └── Async-first documentation: decisions in Confluence, not Slack
threads
│
└── 5. ESTIMATION WAS WRONG?
    → If we estimated 6 months but the work is genuinely 9 months,
       no amount of overtime fixes it.
    → Action: Honest conversation with client. Present data. Propose
options:
       a) Reduce scope (deliver 80% in 6 months)
       b) Extend timeline (100% in 8 months)
       c) Add team (100% in 6 months but costs more — and ramp-up takes 3
weeks)
```

Recovery plan I'd present to the client:

> 'Here's what happened, here's why, and here are three options with trade-offs. I recommend Option A because [reason]. I take accountability for not raising this earlier.'

Key principle: Never surprise the client. If I see a slip coming in Week 6, I raise it in Week 6 — not Week 12 when it's too late to course-correct."

Q3: "How do you manage scope creep without damaging the client relationship?"

Answer:

> "Scope creep is the #1 margin killer in consulting. But handled well, it's actually a revenue opportunity.

My approach (EMBRACE, DON'T FIGHT):

```
PREVENTION:
├── SOW with explicit inclusions AND exclusions
│   "Includes: Migration of 5 services to EKS"
│   "Excludes: Legacy database migration, performance testing"
├── Architecture Decision Records (ADRs) for every new requirement
│   → Creates a paper trail: "This wasn't in the original scope"
└── Change request process agreed in Week 1 (before scope creep starts)

WHEN IT HAPPENS:
├── Step 1: ACKNOWLEDGE (don't say "that's out of scope" immediately)
│   "Great idea. Let me understand the full requirement and assess the
impact."
│
├── Step 2: ASSESS IMPACT (2-day turnaround)
│   "This would add 3 weeks and require 2 additional engineers.
│   It affects Milestone 3 delivery date."
│
├── Step 3: PRESENT OPTIONS (not just "no" or "pay more")
│   Option A: Add to current engagement → +3 weeks, +$X cost
│   Option B: De-prioritize Feature Y to make room → same timeline, same
cost
│   Option C: Phase 2 engagement → handle after go-live (PREFERRED — new
revenue!)
│   "I recommend Option C because it doesn't risk your go-live date,
│   and we can deliver it as a focused 6-week sprint afterward."
│
└── Step 4: DOCUMENT THE DECISION
```

```
    Change request signed by both sides.
    No "we agreed verbally" situations — those always end badly.
```

> **The consulting truth:** 'Out of scope' is the worst phrase in consulting. 'Great addition — let me show you the best way to include it' converts scope creep into scope expansion, which is billable."

---

2. People Management & Team Building

Q4: "You have a team of 15. Two are consistently underperforming, one is a brilliant engineer but a terrible team player. How do you handle each?"

Answer:

> "These are three distinct situations requiring different approaches:

The 2 Underperformers — ROOT CAUSE FIRST:

```
ARE THEY CAPABLE BUT STUCK, OR WRONG FIT?

SCENARIO A: Capable but stuck (most common)
├── Diagnosis: 1:1 conversation — "I've noticed you're struggling with [X].
Help me understand what's blocking you."
├── Common causes:
│   ├── Unclear expectations (they don't know what 'good' looks like)
│   ├── Skill gap on specific technology (never used Terraform before this
engagement)
│   ├── Personal issues (everyone has seasons of struggle)
│   └── Wrong assignment (backend developer doing infrastructure work)
├── Action plan:
│   ├── Pair them with a strong engineer for 2 weeks
│   ├── Set clear, measurable weekly goals (not vague "improve")
│   ├── Daily 15-min check-in (not micromanagement — support)
│   └── Review at 2-week mark: improved? → Great. Not improved? → Escalate.
└── Timeline: 2-4 weeks to decide

SCENARIO B: Wrong fit for the engagement
├── Diagnosis: Despite support, they can't meet the technical bar
├── Action:
```

```
│   ├── Don't keep them on the bench "hoping it gets better"
│   ├── Conversation with Xebia resource manager: "I need to swap [person]
│   │   for someone with [skill]. This is about fit, not failure."
│   ├── Position it positively for the person: "Your strengths are better
│   │   utilized on [other engagement]"
│   └── Replace within 1 week — delay costs the client and the team
└── Key: speed. Every week with a wrong-fit person costs everyone.
```

The Brilliant-But-Toxic Engineer:

```
THE HARDEST MANAGEMENT CHALLENGE:

Step 1: PRIVATE, DIRECT CONVERSATION
"Raj, your technical skills are outstanding. Your Kubernetes expertise
saved us 2 weeks on the migration. But I've received feedback that your
code review comments are harsh, you dismiss others' ideas in meetings,
and two team members have asked not to pair with you. This is a problem
I need you to help me solve."

Step 2: BEHAVIORAL CONTRACT (not vague "be nicer")
├── Specific: "In code reviews, critique code, not the person.
│   Replace 'This is terrible' with 'This could be improved by...'"
├── Measurable: "I want zero complaints from teammates in the next 2 weeks"
├── Observable: "I'll sit in on your next 2 code reviews to model the
behavior"
└── Consequence: "If this doesn't change, I'll need to restructure your role
to
    reduce team interaction — which limits your growth here."

Step 3: IF NO CHANGE IN 3 WEEKS
├── Restructure: move them to a solo track (infrastructure tooling,
automation)
│   where their brilliance adds value but their toxicity is contained
├── If that's not possible: escalate to management for potential removal
└── Hard truth: one toxic person can drive away 3 good people.
    The math never works in favor of keeping them.

KEY PRINCIPLE:
"I never sacrifice team culture for individual brilliance.
A team of 8 collaborative engineers outperforms a team of 10 with 2
toxic members — because the 8 share knowledge, help each other,
and don't dread coming to work."
```

Q5: "How do you build and retain a high-performing team for consulting engagements?"

Answer:

```
MY TEAM BUILDING PHILOSOPHY:

HIRING (who I look for):
├── T-shaped: Deep in ONE area (K8s, Terraform, AWS) + broad curiosity
├── Client-ready: Can explain technical concepts to non-technical people
├── Self-directed: I shouldn't have to assign tasks — they should see work
and pick it up
├── Growth mindset: Will they be better 6 months from now? (past learning
trajectory)
└── Red flag: "I only do [narrow thing]" — consulting requires range

ONBOARDING (first 30 days):
├── Week 1: Shadow me on client calls. See how I handle scope discussions,
demos, escalations
├── Week 2: Own a small deliverable end-to-end. I review and give detailed
feedback
├── Week 3: Lead a client demo (I'm in the room as backup but don't speak)
├── Week 4: Independent delivery. I'm available but not hovering
└── By Day 30: they should feel ownership, not assignment

RETENTION (why people stay):
├── GROWTH: Every 3 months, check: "Are you learning new things?"
│   If not → rotate them to a different client/technology
│   Stagnation is the #1 reason good engineers leave consulting
│
├── VISIBILITY: Credit their work publicly
│   In steering committee: "Anisha designed this solution" (not "the team")
│   In Xebia: nominate for awards, tech talks, blog posts
│
├── AUTONOMY: Trust them with decisions
│   I don't review every PR. I review architecture decisions.
│   Micromanagement kills retention faster than low pay.
│
├── CAREER PATH: Quarterly career conversations
│   "Where do you want to be in 2 years? How does this engagement move you
there?"
│   If this engagement doesn't serve their growth, help them transition
(even to another team)
│
└── COMPENSATION: Advocate fiercely
    If someone is delivering at a senior level but paid at mid-level,
    I escalate to HR with data. People shouldn't have to ask for what they
deserve.
```

> **My track record:** In my last 3 engagements, zero voluntary attrition during the engagement period. 2 engineers I mentored were promoted to Lead Architect within 18 months."

Q6: "How do you mentor senior engineers to become architects?"

Answer:

> "The biggest gap between a senior engineer and an architect isn't technical knowledge — it's **breadth of thinking**. Engineers solve problems. Architects decide which problems to solve.

My mentoring framework:

```
STAGE 1: BROADEN THEIR LENS (Month 1–3)
├── Exercise: "Design Review Shadowing"
│   They sit in my architecture reviews silently. Afterwards:
│   "What did you notice about HOW I asked questions? What did I NOT draw on
the whiteboard?"
│   Learning: Architects ask 'why' and 'what if' before 'how'
│
├── Exercise: "Trade-off Journal"
│   Every week, write one trade-off decision they observed.
│   Format: "We chose X over Y because [context]. If [context changes], we'd
revisit."
│   Learning: There are no right answers — only contextualized decisions.
│
└── Exercise: "Explain to a VP"
    Take a technical concept (e.g., service mesh) and explain it in 2
minutes
    to a business leader with ZERO jargon.
    Learning: If you can't explain it simply, you don't understand it deeply
enough.

STAGE 2: OWN ARCHITECTURE DECISIONS (Month 3–6)
├── Give them a bounded context to architect independently
│   "Design the caching strategy for this module. Present your ADR to the
team."
│   I review BEFORE the presentation, give feedback, let them present.
│
├── Client interaction: they present their designs to client architects
│   I'm in the room but don't rescue them (unless it's going badly)
│
└── Failure budget: They WILL make suboptimal decisions. That's the
learning.
    I only intervene if the decision is:
    - Irreversible (choosing a database is; choosing a logging library
isn't)
    - Compliance-critical (security, data handling)
    - Reputation-affecting (visible to the client's leadership)

STAGE 3: INFLUENCE WITHOUT AUTHORITY (Month 6–12)
├── They lead cross-team architecture discussions
├── They write proposals and RFCs that other teams adopt
```

```
├── They mentor a junior engineer (teaching deepens understanding)
└── Graduation signal: When THEY start asking ME "should we rethink this
approach?"
    instead of waiting for me to catch it — they're thinking like an
architect.
```

---

3. Stakeholder Management & Escalation

Q7: "The client's VP of Engineering bypasses your team and directly assigns work to your developers. How do you handle this?"

Answer:

> "This is extremely common in consulting, and it's destructive if not addressed. It creates invisible scope creep, splits team focus, and makes it impossible to track delivery.

My approach:

```
Step 1: DON'T REACT PUBLICLY (never undermine the VP in front of the team)

Step 2: PRIVATE CONVERSATION WITH THE VP (same day)
"Hey [VP], I noticed you asked Rahul to look at the API gateway issue
directly. I totally understand the urgency — I want to make sure we help
you with that. Can we route requests through me so I can prioritize them
alongside our sprint commitments? That way nothing falls through the cracks
for either of us."

KEY FRAMING:
"I'm trying to help you" (not "you're disrupting my process")

Step 3: SET UP A MECHANISM
├── Shared Slack channel: #healthcloud-requests
│   VP posts requests → I triage within 2 hours
│   Priority tagging: 🔴 urgent (same day) / 🟡 this sprint / 🟢 backlog
├── Weekly priorities alignment (15-min call with VP)
│   "Here's what we're delivering this week. Any changes to priorities?"
└── This gives the VP what they want (fast response) without breaking the
team's flow
```

```
Step 4: IF IT CONTINUES
├── Escalate (gently) to the steering committee
│   Frame: "We want to ensure [VP] gets the fastest response. Currently,
│   direct requests are competing with sprint commitments. Can we agree on
│   a triage process?"
└── Have your Xebia Delivery Manager reinforce the boundary
    (Sometimes it's more effective coming from a peer-level leader)
```

> **Why this matters:** If I let direct assignments happen, my team becomes a body shop, not a consulting partner. The moment we lose control of our own prioritization, we lose the ability to deliver on commitments — and that's when projects fail."

Q8: "Your client CTO and their VP of Engineering disagree on the architecture direction. The CTO wants microservices, the VP wants to keep the monolith. You're caught in the middle. How do you navigate this?"

Answer:

> "This is a political situation disguised as a technical one. My job is to de-politicize it with data.

Approach:

```
STEP 1: UNDERSTAND EACH PERSON'S 'WHY' (separate 1:1s)
├── CTO: "What business outcome are you hoping microservices will deliver?"
│   Usually: "faster feature delivery, independent team scaling, cloud-
native"
│
├── VP Eng: "What's your concern about microservices?"
│   Usually: "we don't have the operational maturity, too many services for
too few people,
│   the monolith works fine, my team can't handle the complexity"
│
└── KEY INSIGHT: They often want the SAME outcome (faster delivery)
    but disagree on the PATH (break everything apart vs. improve what
exists)

STEP 2: CREATE AN OBJECTIVE ASSESSMENT
├── Assess against the team's actual maturity:
```

```
│    ├── Deployment frequency: daily? weekly? monthly?
│    ├── Monitoring maturity: can they debug a distributed system?
│    ├── Team size: enough for independent service ownership?
│    └── Current pain: is the monolith actually the bottleneck?
│
├── Present findings to BOTH together (no separate narratives):
│   "Based on our assessment, your team is at Level 2 maturity on the
│   cloud-native scale. Microservices require Level 4. Here's the gap."

STEP 3: PROPOSE A THIRD OPTION (the bridge)
├── "I recommend a modular monolith as the next step. It gives you:
│    ├── Domain boundaries inside the monolith (CTO gets modularity)
│    ├── No distributed systems complexity yet (VP gets stability)
│    ├── Foundation for future extraction (path to microservices exists)
│    └── Team upskilling during this phase (CI/CD, monitoring, IaC)
│
│   After 6 months, we re-assess: if the team has grown to Level 4,
│   we extract the highest-value bounded context into a microservice."
│
└── This works because:
     ├── CTO sees a path to their vision (not blocked, just phased)
     ├── VP sees operational risk managed (not reckless modernization)
     └── I provided data, not opinion — neither person "lost"

STEP 4: DOCUMENT THE DECISION
├── ADR co-signed by CTO and VP Eng
├── Decision: Modular monolith NOW, microservices extraction in Phase 2
├── Review criteria: re-assess at 6 months based on [specific metrics]
└── This prevents the debate from re-opening every week
```

> **The leadership lesson:** When two senior leaders disagree, the worst thing you can do is pick a side. The best thing is to reframe the debate from 'A vs. B' to 'A then B' or 'C that satisfies both.' Your value as a consultant is being the neutral party with data."

---

4. Technical Decision-Making Under Constraints

Q9: "The client wants to go live in 3 months. Your architecture assessment says they need 6 months of foundation work. How do you reconcile this?"

Answer:

```
THE REALITY: You almost NEVER get the timeline you need in consulting.
The skill is deciding what's ESSENTIAL vs. what's IDEAL.

MY FRAMEWORK: "3-MONTH VIABLE vs. 6-MONTH IDEAL"

ESSENTIAL (Must have for Day 1):                    DEFER (Phase 2 after go-
live):
├── EKS cluster with private subnets                ├── Multi-region DR
├── CI/CD pipeline (build → deploy)                 ├── Chaos engineering
├── Basic monitoring (CloudWatch + PagerDuty)        ├── FinOps optimization
├── IAM + RBAC (no public endpoints)                ├── Service mesh (Istio)
├── Database encryption (KMS)                        ├── Advanced
observability (OpenTelemetry)
├── Single-environment Terraform                     ├── Multi-account
landing zone
├── 3 core services migrated                         ├── Remaining 15
services
└── Runbook for top 5 failure scenarios              └── Comprehensive
documentation

WHAT I TELL THE CLIENT:
"We can go live in 3 months with a 'Minimum Viable Platform.' Here's
what that includes, and here's what it doesn't. The items we defer are
not optional — they're Phase 2, starting the week after go-live.

The risk of this approach: we'll operate without multi-region DR for
3 months. If your RPO requirement is < 15 minutes, we need to either:
  a) Extend to 5 months (add DR in Phase 1)
  b) Accept RPO = 4 hours for the first 3 months (multi-AZ, single-region)
  c) Run the old system as passive DR during Phase 2

I recommend option C — it's the safest for your timeline."

WHAT I TELL MY TEAM:
"We're cutting scope, not quality. Everything we ship in 3 months must
be production-grade — secure, tested, documented. We're NOT shipping
hacks and fixing them later. We're shipping less, done right."
```

Q10: "The client has a team of 40 developers but zero Kubernetes experience. How do you plan the skill transformation alongside the technical transformation?"

Answer:

```
PARALLEL TRACKS — TECHNOLOGY AND PEOPLE TRANSFORM TOGETHER:

TRACK 1: PLATFORM TEAM (Month 1–2)
├── Select 3–5 strongest engineers from client team → "Platform Champions"
├── Intensive bootcamp: 2 weeks of hands-on K8s + Terraform + CI/CD
│   NOT classroom training — pair programming on the ACTUAL project
│   "We're not teaching Kubernetes. We're building YOUR platform together."
├── They own the platform from Day 1 (with Xebia pair-programming)
├── After 2 months: they can independently provision environments,
│   debug pod issues, and review Terraform PRs
└── These 5 become the internal support network for the other 35

TRACK 2: APPLICATION TEAMS (Month 2–4)
├── "Golden Path" approach:
│   ├── Xebia builds: reference microservice (Dockerfile, Helm chart, CI/CD
pipeline)
│   ├── Document: "How to deploy your service in 30 minutes" (step-by-step)
│   ├── Application teams follow the golden path — don't build their own
│   └── Creativity constrained to APPLICATION code; infrastructure is
standardized
│
├── Developer Experience metrics I track:
│   ├── Time to first deployment: Day 1 target = < 4 hours for a new dev
│   ├── Time from commit to production: target = < 30 minutes
│   └── Support tickets to platform team: should decrease week-over-week
│
└── Learning model: "See one, do one, teach one"
    Week 1: Developer shadows Xebia engineer deploying a service
    Week 2: Developer deploys with Xebia engineer watching
    Week 3: Developer deploys independently, Xebia reviews
    Week 4: Developer helps ANOTHER developer deploy (now they're teaching)

TRACK 3: OPERATIONS TEAM (Month 3–5)
├── Embed Xebia SRE with client ops team
├── Joint on-call rotation (Xebia primary, client shadow → then swap)
├── Runbook creation: "If X happens, do Y" for top 20 scenarios
├── Incident simulation: monthly game-day exercises
└── By Month 5: client ops team handles 80% of incidents independently

EXIT CRITERIA (when Xebia can step back):
├── Platform team can handle K8s upgrades independently
├── Application teams can deploy new services without asking for help
├── Ops team can handle P1 incidents without calling Xebia
├── All runbooks written and validated
└── Internal champion network can train new joiners
```

---

5. Pre-Sales, Revenue & Commercial Awareness

Q11: "As a Principal Architect, how do you contribute to Xebia's revenue growth?"

Answer:

> "A Principal Architect who doesn't think about revenue is a Senior Engineer with a title. At Xebia, my architecture decisions ARE business decisions.

My revenue contribution model:

```
1. ENGAGEMENT EXPANSION (biggest lever)
├── Every engagement should naturally identify the NEXT engagement
├── Example: During cloud migration, I identify:
│   ├── "Your monitoring is basic — we can build an observability platform"
(6-week engagement)
│   ├── "You have no DR strategy — we can design and implement one" (8-week
engagement)
│   ├── "Your FinOps is nonexistent — we can save you 30% on cloud" (4-week
engagement)
│   └── Total: 1 engagement → 3 follow-on engagements = 3x revenue
multiplier
│
├── HOW I do this without being "salesy":
│   I include a "Recommendations & Next Steps" section in every milestone
report.
│   It's positioned as advice, not a sales pitch.
│   "Based on our work this quarter, we've identified 3 areas of technical
debt
│   that pose risk to your platform. Here's our recommended remediation
plan."
│   The client ASKS for the next engagement.

2. REUSABLE ACCELERATORS (efficiency lever)
├── I build once, use across engagements:
│   ├── Terraform module library (AWS/Azure reference architecture)
│   ├── Kubernetes security baseline (our healthcloud-multiregion-dr
project!)
│   ├── CI/CD pipeline templates (GitHub Actions for Terraform + Docker +
ArgoCD)
│   ├── DevOps automation scripts (security scanner, compliance checker)
│   └── Interview-ready architecture diagrams for pre-sales
│
├── Impact: New engagement setup goes from 3 weeks to 3 days
│   → Xebia delivers faster → higher margin → client sees value faster
│   → Xebia can bid at lower cost (accelerators reduce effort) → more wins

3. THOUGHT LEADERSHIP (brand lever)
├── Technical blog posts → attract inbound leads
├── Conference talks → position Xebia as a thought leader
├── Client case studies (anonymized) → used in pre-sales pitches
├── Architecture workshops → "free" value that converts to paid engagements
```

```
│   "Let us run a 1-day architecture assessment. No commitment."
│   → 70% of these convert to paid engagements (industry average)

4. PRE-SALES INVOLVEMENT
├── Technical solutioning: write the technical approach in proposals
├── Effort estimation: work with delivery managers to estimate realistically
│   (Architects who estimate poorly lose the company money or credibility)
├── Client presentations: join pre-sales calls as the "technical expert"
│   Role: answer deep technical questions, build client confidence
└── Competitor differentiation: "Here's why our approach is better than
    what [competitor] proposed" (technical depth, not price)
```

Q12: "How do you estimate effort and pricing for a cloud transformation engagement?"

Answer:

```
MY ESTIMATION FRAMEWORK:

STEP 1: DECOMPOSE INTO WORK STREAMS
├── Work Stream 1: Platform Foundation (IaC, CI/CD, K8s, Security)
├── Work Stream 2: Application Migration (per service: assess + migrate +
test)
├── Work Stream 3: Data Migration (schema, data, cutover)
├── Work Stream 4: Observability & Operations (monitoring, alerting,
runbooks)
├── Work Stream 5: Security & Compliance (audit, remediation, certification)
└── Work Stream 6: Knowledge Transfer & Documentation

STEP 2: ESTIMATE PER WORK STREAM (3-point estimation)
For each work stream:
├── Optimistic: everything goes right, team is experienced
├── Most Likely: normal friction, some unknowns
├── Pessimistic: significant blockers, client dependencies delayed
└── Use PERT: (O + 4×ML + P) / 6 = Expected duration

STEP 3: TEAM COMPOSITION
├── Principal Architect (me): 50% utilization (architecture + client
management)
├── Lead Engineers (2): 100% utilization (technical execution)
├── Senior Engineers (4-6): 100% utilization (delivery)
├── DevOps/SRE (2): 100% utilization (platform + operations)
└── Project Manager (1): manage delivery, reporting, logistics

STEP 4: RISK BUFFER
├── Known unknowns: add 15% buffer
├── Client dependency risk: add 10% (they're always late with
access/decisions)
├── Scope growth: assume 10% natural scope expansion
└── Total: estimate × 1.35 = proposal estimate
```

```
STEP 5: PRICING MODEL
├── Time & Materials: better for uncertain scope (discovery, R&D)
│   Rate: per-person per-month × team size × duration
├── Fixed Price: better for well-defined scope (migration of X services)
│   Price = estimate + risk buffer + margin
├── Outcome-Based: rare but powerful
│   "Pay us $X/month. We guarantee your deployment frequency reaches
│   daily within 6 months or we extend at no cost."
└── MY PREFERENCE: T&M with milestone-based caps
    Client pays per month but with agreed milestones.
    If we deliver faster, we earn trust. If we slip, we absorb the overrun
(within buffer).
```

---

6. Risk Management & Governance

Q13: "A critical production incident happens at the client during your engagement. Their system — which you partially built — goes down at 2 AM. Walk me through your response."

Answer:

```
INCIDENT RESPONSE — THE FIRST 60 MINUTES:

MINUTE 0-5: ACKNOWLEDGE & MOBILIZE
├── Phone buzzes at 2 AM. PagerDuty alert.
├── First action: Acknowledge the alert (SLA: < 5 min)
├── Join the war room (Slack channel / Zoom bridge)
├── Quick assessment: What's the BLAST RADIUS?
│   ├── Full outage? Partial degradation? One service?
│   └── Is customer data at risk? (HIGHEST priority for healthcare/fintech)
└── Communicate: "I'm online. Assessing now. Will update in 10 minutes."

MINUTE 5-15: DIAGNOSE
├── Check the dashboards (CloudWatch, Grafana, PagerDuty timeline)
├── Recent deployments? (Most incidents = recent change. Check ArgoCD
history.)
├── Infrastructure change? (Terraform apply in the last 24 hours?)
├── External dependency? (AWS status page, third-party API status)
├── Traffic spike? (DDoS or legitimate surge?)
```

```
└── Form a hypothesis within 15 minutes

MINUTE 15-45: MITIGATE
├── If recent deployment → ROLLBACK FIRST, debug later
│   argocd app rollback [app] — don't spend 30 min debugging when rollback
takes 2 min
├── If infrastructure → check resource exhaustion (CPU, memory, connections,
IOPS)
│   Scale up or restart affected components
├── If data issue → DO NOT attempt data fixes at 2 AM. Isolate the problem.
│   Redirect traffic to read-only mode if possible
└── Goal: Restore service first. Root cause later.

MINUTE 45-60: COMMUNICATE
├── Update to stakeholders: "Service restored at [time]. Root cause
investigation
│   underway. Full post-mortem report in 48 hours."
├── Client CTO doesn't need to know every technical detail at 2 AM
│   They need: "Are we up? Is data safe? What's the plan?"
└── Internal Xebia update: escalate to delivery manager if client
relationship risk

POST-INCIDENT (within 48 hours):
├── Blameless post-mortem
│   ├── Timeline: what happened, when, who did what
│   ├── Root cause: 5-whys analysis
│   ├── Contributing factors: what made it worse or harder to diagnose
│   ├── Action items: preventive measures with owners and deadlines
│   └── What went well: acknowledge fast response, effective teamwork
├── Monitoring gaps: what SHOULD have alerted earlier?
├── Runbook update: add this scenario to the operations runbook
└── Present findings to client (transparency builds trust)

WHAT I NEVER DO:
├── Blame my team members in front of the client
├── Speculate on root cause before I have data
├── Promise "this will never happen again" (it will — the question is how
fast we detect and recover)
└── Hide that our code contributed to the issue (if it did)
```

Q14: "How do you handle a situation where the client asks you to cut corners on security to meet a deadline?"

Answer:

> "This is a non-negotiable for me — and it's a conversation I've had multiple times.

My response framework:

```
STEP 1: UNDERSTAND WHAT THEY'RE ACTUALLY ASKING
├── "Cut corners on security" usually means:
│   ├── Skip security testing (pen test, Trivy scan)
│   ├── Use default credentials "for now"
│   ├── Open security groups to 0.0.0.0/0 "temporarily"
│   ├── Skip encryption "because it adds latency"
│   └── Deploy without WAF or IAM policies
│
├── Some of these are NEVER acceptable:
│   ├── Default credentials → No. This takes 5 minutes to fix.
│   ├── Open security groups → No. Never even temporarily.
│   └── Skip encryption on PHI → No. This is a compliance violation.
│
└── Some can be STAGED:
    ├── Skip pen testing → OK for staging, but mandatory before prod
    ├── Simplified IAM → Broad roles now, fine-grained in Phase 2
    ├── Basic WAF → AWS managed rules now, custom rules later
    └── These are RISK ACCEPTANCE decisions, not security "shortcuts"

STEP 2: QUANTIFY THE RISK
"If we skip [security measure], here's the exposure:
 - Data breach: Average cost in healthcare = $10.9M (IBM 2025 report)
 - Compliance fine: HIPAA violation = up to $1.5M per occurrence
 - Brand damage: immeasurable
 - Insurance: your cyber insurance policy likely requires [this measure]

 The 2 weeks we save now could cost $10M+ later."

STEP 3: PROPOSE AN ALTERNATIVE
"Let's not skip security — let's SIMPLIFY it:
 Instead of a full pen test (2 weeks), let's do:
 - Automated scanning: Trivy + tfsec in CI (adds 5 minutes to pipeline)
 - Basic WAF: AWS managed rules (deploy in 1 hour)
 - Simplified IAM: least-privilege at service level (not per-API)

 This gives us 80% of the security in 20% of the time.
 We schedule the full pen test for Month 1 post-launch."

STEP 4: IF THEY INSIST
├── Put it in writing: "Client has accepted the risk of operating without
│   [measure] until [date]. Risk acknowledged by [VP name] on [date]."
├── Escalate internally to Xebia delivery lead
├── If it's a compliance violation (not just best practice):
│   "I cannot implement this in a way that violates [HIPAA/PCI/GDPR].
│   This isn't a judgment call — it's a legal requirement."
└── Be prepared to lose the argument gracefully on RISK ACCEPTANCE
    but NEVER on COMPLIANCE VIOLATION
```

---

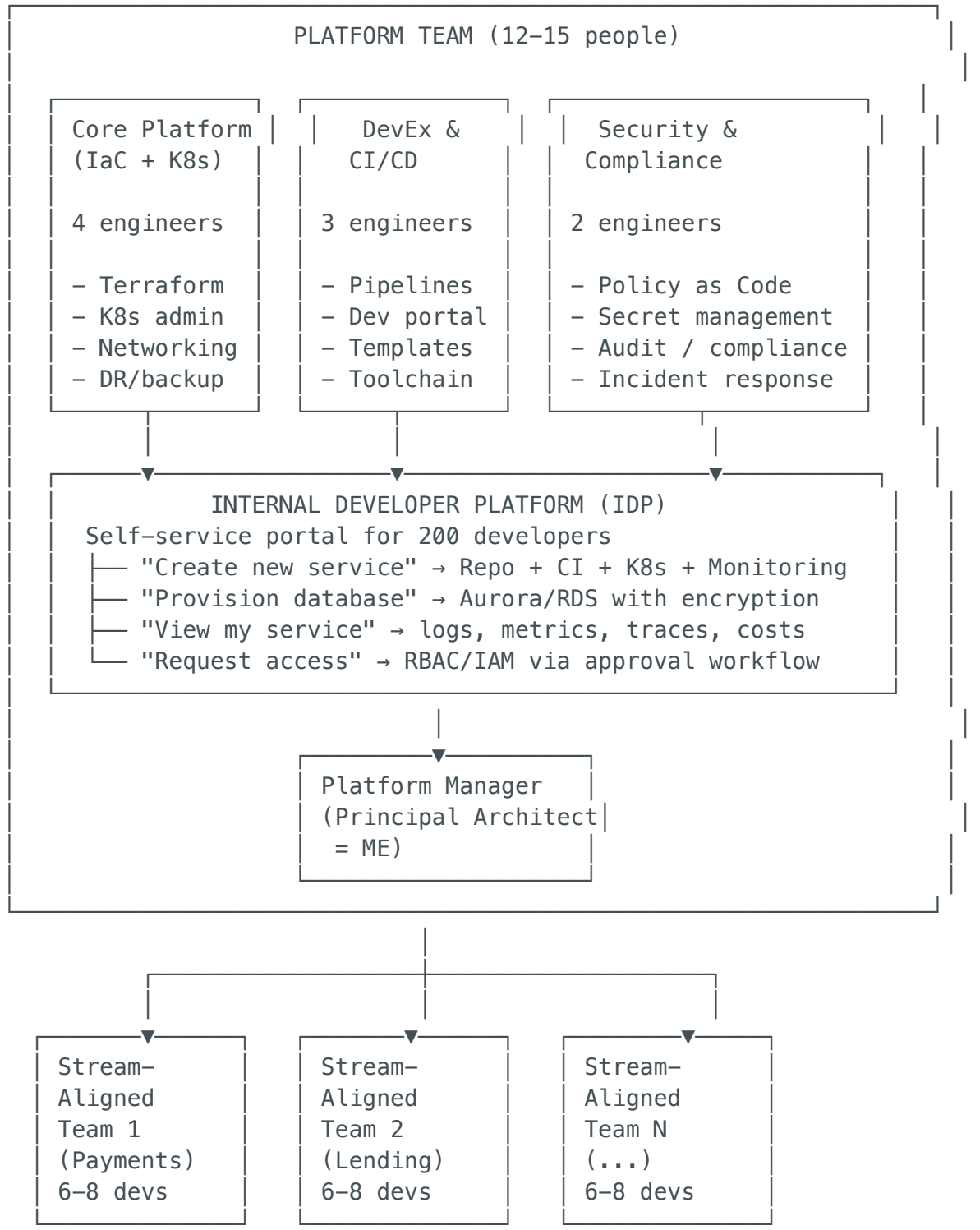
7. Organizational Design & Process

Q15: "How would you structure a platform engineering team for a 200-developer organization?"

Answer:

```
PLATFORM TEAM ORGANIZATIONAL DESIGN:

TEAM TOPOLOGY (Team Topologies model):
```




```
INTERACTION MODE:
```

```
Platform team is "X-as-a-Service" — NOT a gatekeeper.
├── Stream-aligned teams DON'T file tickets for environments
│   They use the self-service portal.
│   If they need to file a ticket, the platform has FAILED.
│
├── Platform team success metric:
│   "What % of developer requests are self-served?"
│   Target: > 80%
│
└── Engagement model:
    Platform team holds office hours (not on-call for stream teams)
    Deep support through "Enabling Teams" temporarily embedded in stream
teams
```

---

8. Scenario-Based Situational Judgement

Q16: "A key team member gets a competing offer with 40% more pay during a critical phase of the engagement. They come to you first. What do you do?"

Answer:

> "This is a test of whether I value the person or just the delivery.

My response:

```
IMMEDIATE (same conversation):
├── "Thank you for telling me before deciding. That shows trust, and I
respect it."
├── "Before I react — what's driving this? Is it purely the money,
│   or are there other factors?" (Often: growth, challenge, culture,
manager)
├── Listen. Don't counter immediately. Understand the WHOLE picture.
└── "Can I have 48 hours to see what I can do? Don't accept yet."

WITHIN 48 HOURS:
├── If money is the primary driver:
│   ├── Talk to Xebia HR: "I have a critical retention risk. What's
possible?"
```

```
│   ├── Present what's possible: salary increase, retention bonus, role
upgrade
│   ├── Be honest: "If we can't match 40%, here's what we CAN offer:
│   │   15% raise + promotion to Lead + conference budget + interesting next
engagement"
│   └── Sometimes the package (growth + trust + culture) > raw compensation
│
├── If growth/challenge is the driver:
│   ├── "What would make you excited to stay? Let's design that together."
│   ├── Concrete offers: Lead the next engagement, architecture ownership,
│   │   client-facing role, technology you want to learn
│   └── This often works because the other company is unknown; we offer
certainty
│
└── If they've already decided to leave:
    ├── "I respect your decision. Let's make the transition smooth."
    ├── Negotiate notice period: "Can you stay 4 weeks instead of 2?
    │   I need time to find coverage for the engagement."
    ├── Knowledge transfer plan: document what's in their head
    └── Part on good terms: "If it doesn't work out, the door is open."
        (People who leave well sometimes come back)

AFTER THEY DECIDE:
├── If they stay: deliver on EVERY promise. Nothing kills retention faster
│   than unkept promises after a counter-offer.
├── If they leave: no guilt, no passive-aggression. Professional.
│   Message to team: "[Person] is moving on. We wish them well.
│   Here's how we're redistributing work."
└── Self-reflection: "What should I have done 3 months ago to prevent this?"
    Was I checking in on their growth? Compensation? Happiness?
```

Q17: "You discover that a senior engineer on your team has been inflating their timesheet — billing the client for hours they didn't work. What do you do?"

Answer:

> "This is an ethics situation — there's no 'manage around it.' It's zero tolerance.

```
STEP 1: VERIFY (don't accuse based on suspicion)
├── Review timesheets vs. actual deliverables, git commits, meeting
attendance
├── Talk to the person's pair/teammate: "Did Raj work on the auth module
last Thursday?"
├── Check Jira/PR activity for the disputed dates
└── Is there a pattern (recurring) or a one-time discrepancy?
```

```
STEP 2: IF ONE-TIME DISCREPANCY
├── It might be an honest mistake (logged wrong date, confused project
codes)
├── Private conversation: "I noticed your timesheet shows 8 hours on
Thursday
│   for the auth module, but there's no activity. Can you help me
understand?"
├── If reasonable explanation → note it and move on
└── Update process: timesheets approved by me weekly going forward

STEP 3: IF CLEAR, REPEATED INFLATION
├── This is fraud. It damages Xebia's reputation and the client's trust.
├── Immediate actions:
│   ├── Document the evidence (screenshots, timestamps, git logs)
│   ├── Escalate to Xebia HR and Delivery Manager SAME DAY
│   ├── Do NOT confront the person alone (HR should be involved)
│   └── Xebia leadership decides the personnel action (not me alone)
├── Client communication:
│   ├── Proactive disclosure: "We identified a billing discrepancy.
│   │   We're crediting $X to your account and have taken corrective
action."
│   ├── Don't name the person (privacy)
│   ├── Show what controls we're implementing to prevent recurrence
│   └── Transparency here BUILDS trust; covering it up DESTROYS it
└── Team message (after person is removed):
    "[Person] is no longer on the engagement. I can't share details,
     but I want to reinforce: integrity is non-negotiable."

KEY PRINCIPLE:
"One dishonest person can destroy years of client trust.
Xebia's reputation is worth more than any individual contributor."
```

---

9. Leadership & Strategy

Q18: "Where do you see the future of cloud architecture and DevOps in the next 3-5 years, and how should Xebia position itself?"

Answer:

```
MY PREDICTIONS & XEBIA'S POSITIONING:

1. PLATFORM ENGINEERING REPLACES DEVOPS (2025–2027)
```

```
├── DevOps as a "team" is dying. Platform as a PRODUCT is rising.
├── Every enterprise will need an Internal Developer Platform (IDP)
├── Xebia positioning: Build a "Platform-in-a-Box" accelerator
│   Pre-built IDP with IaC, CI/CD, observability, developer portal
│   Deploy in 4 weeks, not 4 months. Premium offering.

2. AI-AUGMENTED OPERATIONS (2025–2028)
├── LLMs for incident diagnosis, code review, infrastructure optimization
├── "Claude Code for DevOps" (what we just built!) becomes mainstream
├── Xebia positioning: AI-powered DevOps practice
│   Sell AI-augmented SRE services, automated compliance, intelligent
alerting
│   This is a NEW service line, not an add-on

3. MULTI-CLOUD BECOMES DEFAULT (2026–2028)
├── Not by choice — by acquisition, regulation, and risk management
├── Healthcare (HIPAA), finance (RBI), government (data sovereignty) drive
this
├── Xebia positioning: Multi-cloud reference architectures (like our
healthcloud project)
│   AWS + Azure expertise is rare. Most consultancies do one well, not both.

4. FINOPS + GREENOPS BECOME C-SUITE PRIORITIES (2026+)
├── Cloud bills are now the #3 IT expense after people and facilities
├── Carbon footprint of cloud will face regulation in EU
├── Xebia positioning: Sustainability-aware cloud architecture
│   "We don't just optimize for cost — we optimize for cost AND carbon"

5. SECURITY SHIFTS FULLY LEFT (NOW)
├── DevSecOps is no longer optional — it's expected in every engagement
├── Zero-trust architecture becomes the default, not the premium option
├── Xebia positioning: Security is embedded in every engagement,
│   not a separate "security review" engagement

STRATEGIC RECOMMENDATION FOR XEBIA:
├── Invest in accelerators (reusable IP): reference architectures, Terraform
modules,
│   compliance frameworks, AI tools
├── Hire "T-shaped" architects who can bridge business and tech
├── Build a "Xebia Architecture Academy" for talent development
└── Position as "premium consulting + IP" not "staff augmentation"
    (Accenture does staffaug; Xebia should do OUTCOME-based consulting)
```

Q19: "What's your leadership philosophy? How would you describe your leadership style?"

Answer:

> "My leadership style is **'servant-leader with high standards.'** I serve the team by removing blockers, creating clarity, and shielding them from organizational noise. But I hold high standards for delivery quality, communication, and professional growth.

In practice:

```
WHAT MY TEAM WOULD SAY ABOUT ME:

✅ "He gives us enough context to make our own decisions"
   → I share the WHY behind decisions, not just the WHAT.
   → If someone disagrees with my architecture choice and has a better
     argument, I change my mind — publicly.

✅ "He shields us from politics but keeps us informed"
   → Client escalations, scope disputes, budget pressures — I handle these.
   → But I share: "FYI, the client is concerned about timeline.
     Here's what I told them. Here's what we need to deliver."

✅ "He gives feedback directly, not passively"
   → I don't wait for annual reviews. If your code review was great,
     I tell you that day. If your client presentation was weak,
     I tell you that day — privately, with specific suggestions.

✅ "He invests in our growth, not just the project's success"
   → I've advocated for promotions, salary raises, and conference
     sponsorships for team members even when it wasn't "my job."

⚠️ "He can be impatient when people don't prepare"
   → Fair feedback. I expect people to come to architecture discussions
     having READ the context. "I didn't get time to look at it" frustrates
me.
   → I'm working on this — not everyone processes information the same way.
```

---

10. Culture, Growth & Self-Awareness

Q20: "What's the biggest mistake you've made as a leader, and what did you learn?"

Answer:

> "Early in my architect career, I designed a beautiful microservices architecture for a client with 8 developers. 12 services, event-driven, CQRS, the works. Technically elegant. Organizationally impossible.

> The team couldn't maintain 12 services. Deployments took 2 hours because 6 services had to be deployed in order. Debugging required correlating logs across 12 systems. Within 3 months, the team was spending more time on infrastructure than features.

> **What I learned:**

```
1. ARCHITECTURE MUST MATCH ORGANIZATIONAL CAPACITY
   Conway's Law isn't a suggestion — it's a natural law.
   8 developers ≠ 12 services. 8 developers = 3-4 services max.

2. COMPLEXITY IS A COST, NOT A FEATURE
   Every architectural pattern (CQRS, event sourcing, saga) has
   an operational tax. Before adding a pattern, ask:
   "Does this team have the operational maturity to run this?"

3. THE BEST ARCHITECTURE IS THE ONE THE TEAM CAN OPERATE
   Not the one that wins awards. Not the one on the conference slide.
   The one that the client's team can debug at 2 AM after I've left.

4. MY EGO WAS INVOLVED
   I wanted to build something impressive. I should have built
   something appropriate. That's the difference between a technologist
   and an architect.
```

> I fixed it: we consolidated to 4 services (domain-aligned modular services). Deployment time dropped to 15 minutes. The team could actually operate it. I documented the decision and the mistake in an ADR — for the next architect who's tempted to over-engineer."

Q21: "Rate yourself on these dimensions and explain."

Dimension	Rating	Explanation
Technical Depth (AWS/Azure/K8s)	8.5/10	"Deep hands-on across AWS, Azure, Kubernetes. I can debug at the kernel level. Not

Dimension	Rating	Explanation
		a 10 because nobody is — the ecosystem moves too fast."
Architecture & System Design	9/10	"This is my craft. I've designed systems serving millions of users across multiple clouds. I think in trade-offs, not absolutes."
People Management	8/10	"I've led teams of 15+, mentored architects, handled underperformers. Still growing in delegation — I sometimes jump in when I should let others learn through struggle."
Client/Stakeholder Management	8.5/10	"I can present to CTOs and pair-program with junior devs in the same day. I've recovered failing engagements and expanded accounts."
Commercial/Business Awareness	7.5/10	"I understand P&L, engagement economics, and upsell. But I haven't run a practice P&L or owned a revenue target. That's my growth area."
Communication (written + verbal)	8.5/10	"Strong at explaining complex topics simply. Good at written ADRs and proposals. Working on being more concise in meetings."
Conflict Resolution	8/10	"I handle client disagreements and team conflicts directly. My bias is toward harmony, which sometimes delays necessary confrontations by a week."

11. Rapid-Fire Managerial Questions

Question	Strong Answer
What's your biggest pet peeve about consulting?	"When the engagement becomes body shopping — clients using us as additional hands instead of leveraging our expertise. I always push to stay in an advisory + delivery role, not just execution."

Question	Strong Answer
How do you handle burnout in your team during a high-pressure engagement?	"I watch for the signs: PRs getting sloppy, people going quiet in standups, working weekends. My intervention: redistribute work, cancel low-value meetings, enforce a 'no Slack after 8 PM' rule for 2 weeks. And I check in 1:1."
What's the hardest conversation you've had with a client?	"Telling a client that their internal 'cloud expert' was actually the bottleneck. Their CTO had hired this person specifically for the transformation, and their ego was blocking our recommendations. I presented data from sprint velocity before/after their involvement, and proposed restructuring their role to 'technology strategy' instead of 'execution.'"
How do you say no to your manager?	"With data and an alternative. 'I understand the ask. Here's why I think it's risky, and here's what I'd propose instead. If you still want me to proceed with the original plan, I will — but I want to flag the risk.'"
What would you change about your current/last organization?	"More investment in reusable IP and internal tools. Every engagement starts from scratch — we should have a library of accelerators that lets us deliver 2x faster."
How do you handle a team member who's technically strong but doesn't document anything?	"I make documentation a DEFINITION OF DONE, not a nice-to-have. PR without updated README = PR not ready for review. Architecture without ADR = architecture not approved. Then I review and give feedback on docs like I do on code."
Tell me about a time you failed at managing a stakeholder.	"I once escalated a client issue to the steering committee without giving the VP a heads-up first. He felt blindsided. The issue got resolved, but I damaged a relationship. Lesson: always pre-wire stakeholders before escalating. Never surprise anyone in a meeting."
How do you decide when to be hands-on vs. hands-off?	"Hands-on for: architecture decisions, P1 incidents, client presentations, code reviews on critical paths. Hands-off for: sprint planning, daily standups (team runs these), task

Question	Strong Answer
	assignment, technology POCs (delegate to senior engineers)."
If I gave you a poorly performing team tomorrow, what's Day 1?	"Listen. I'd have a 30-min 1:1 with every person. 'What's working? What's broken? What would you change if you could?' Usually the team knows exactly what's wrong — they just haven't been asked or heard."
What separates a good architect from a great one?	"A good architect designs great systems. A great architect designs great systems that great teams can build, operate, and evolve after the architect leaves."

12. Questions YOU Should Ask the L3 Panel

About the Role & Expectations:

1. "*What does success look like for a Principal Architect at Xebia at the 12-month mark? What would make you say 'this hire was a home run'?*"
2. "*How much of my time is expected on delivery vs. pre-sales vs. practice building?*"
3. "*Do Principal Architects at Xebia typically lead single large engagements, or advise across multiple simultaneous engagements?*"

About Team & Culture:

4. "*What's the typical team composition I'd be working with — how many experienced architects vs. junior engineers?*"
5. "*How does Xebia handle situations where a client's culture or ethics conflicts with Xebia's values?*"

About Business & Growth:

6. "*What are Xebia's growth priorities for the next 2 years — new geographies? New industries? New service lines?*"

7. "*Is there an expectation for Principal Architects to contribute to business development or just technical delivery?*"
8. "*What's the revenue target or engagement target for the architecture practice this year?*"

About Your Future:

9. "*What does the career path beyond Principal Architect look like — Practice Lead? VP Engineering? Distinguished Engineer?*"
10. "*If I bring IP and accelerators (like the Terraform modules and reference architectures I've built), how does Xebia value and reward that?*"

Pre-Interview Power Prep

48 Hours Before

- ☐ Re-read L1 doc (skim 70 Q&As) and L2 doc (skim 14 Q&As)
- ☐ Practice 3 STAR stories out loud (3 min each, timed):
  - Story: Project turnaround (from L2, Q Story 1)
  - Story: Saying no to a client (from L2, Q Story 2)
  - Story: NEW — Managing underperformance or team conflict
- ☐ Review Xebia's recent LinkedIn/blog posts — reference them in the interview
- ☐ Prepare 3 engagement expansion examples from your experience
- ☐ LinkedIn-stalk the interviewer — find common ground

Day Of

- ☐ **Opening energy:** Confident but not arrogant. "I'm excited to discuss how I can contribute to Xebia's growth."
- ☐ **When you don't know:** "I haven't faced that specific situation, but here's how I'd approach it..." (Never fake experience)
- ☐ **Close strong:** "This conversation has reinforced my excitement about Xebia. The combination of engineering excellence, client diversity, and the opportunity to build a practice is exactly where I want to be. I'm ready to contribute from Day 1."

The L3 Meta-Principle

> **L1 asks:** Can you code and design? **L2 asks:** Can you architect and consult? **L3 asks:** Can we trust you with a ₹15 Cr engagement, a 15-person team, and a Fortune 500 client?

The answer they want to hear (through your answers, not explicitly):

"Yes. I can own the outcome — technically, commercially, and organizationally. I can lead when things are going well, and I can lead especially when they're not. I build teams, grow revenue, and deliver results that make clients want to work with Xebia again."

Prepared for: Pushparaj Naik | Role: Principal Architect — Xebia | Round: L3 Techno-Managerial Builds on: L1 (70 Q&As) + L2 (14 Q&As + Design Sessions) = Total prep: 110+ Q&As across 3 rounds Prepared: June 2026